

Programming and Algorithms

Aula 15: Ordenação e Buscas em Sequências

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Programa da Aula

- Por que ordenar itens de sequências?
- Construção de algoritmos para ordenação de sequências mutáveis
- Outros algoritmos de ordenação
- Recursos integrados ao Python para ordenação
- Como buscar itens em sequências
- Algoritmos de buscas linear e binária

Introdução

- A ideia principal continua a ser manipular dados em sequências de maneira eficaz.
- Quando trabalhamos com uma grande quantidade de dados normalmente nos deparamos com dois tipos de operações:
  - Busca por dados específicos
  - Ordenação dos dados por algum critério

Ordenação

- Ordenar uma sequência significa reorganizá-la de acordo com alguma ordem.
- E por que devemos ordenar sequências?
  - **Sequências ordenadas tornam as buscas por seus itens mais eficientes.**

Troca de itens de uma sequência: Criando uma lista e trocando dois de seus itens

```
lista = [10, 40, 30, 20, 50]
temp = lista[1]
lista[1] = lista[3]
lista[3] = temp
lista
[10, 20, 30, 40, 50]
```

Veja no Python Tutor

Ordenação

- Podemos melhorar a codificação anterior utilizando **atribuição paralela**, que pode ser feita com o uso de **desempacotamento**

```
lista = [10, 40, 30, 20, 50]
lista[1], lista[3] = lista[3], lista[1]
lista
[10, 20, 30, 40, 50]
```

Mas ainda pode-se melhorar a codificação

Ordenação

```
def troca(s, i, j):
    s[i], s[j] = s[j], s[i]
```

A função acima recebe como argumentos uma sequência mutável s e índices i e j que sejam válidos em s, fazendo a troca s[i] com s[j].

Depois de editar a função troca e executar, teste no shell.

```
========== RESTART: /Users/Ettore/L
>>> lista = [10, 40, 30, 20, 50]
>>> troca(lista,1, 3)
>>> lista
[10, 20, 30, 40, 50]
>>>
```

Empurrando o item máximo de uma sequência

Como preparação para o algoritmo de ordenação por flutuação, veremos o funcionamento da sua operação núcleo que é baseada em empurrar (flutuar) o item máximo de uma sequência para sua última posição. O algoritmo compara todos os pares vizinhos da sequência e, se o primeiro item do par for maior do que o segundo, ele realiza uma troca entre eles.

```
def troca(s, i, j):
    s[i], s[j] = s[j], s[i]

lista = [40, 30, 20, 50, 10]
for i in range(len(lista)-1):
    if lista[i] > lista[i+1]:
        troca(lista, i, i+1)

print(lista)
```

Empurrando o item máximo de uma sequência

```
lista = [40, 30, 20, 50, 10]
for i in range(len(lista)-1):
    if lista[i] > lista[i+1]:
        troca(lista, i, i+1)
print(lista)
```

A variável i assumirá os índices da lista, do primeiro até o seu penúltimo pois a condição if está baseada na comparação de itens vizinhos.

Caso i assumisse o valor do último índice não conseguiríamos testar o par de números. Vamos ver o teste de mesa da codificação anterior:

Empurrando o item máximo de uma sequência

```
lista = [40, 30, 20, 50, 10]
for i in range(len(lista)-1):
    if lista[i] > lista[i+1]:
        troca(lista, i, i+1)
print(lista)
```

Tabela 11.1: Teste de mesa da Codificação 11.5.

lista					i	lista[i]	lista[i+1]	troca?
[0]	[1]	[2]	[3]	[4]				
40	30	20	50	10	0	40	30	True
30	40	20	50	10	1	40	20	True
30	20	40	50	10	2	40	50	False
30	20	40	50	10	3	50	10	True
30	20	40	10	50				

Note que o item máximo foi empurrado para o final da lista!

Melhorando a codificação

- A ideia é criar um função que receberá como argumentos uma sequência mutável s e seu **tamanho lógico** n.
- A função empurrará o item máximo da sequência s para o final dela, ou seja, para a posição de índice n-1.
- **n é o tamanho lógico de s, pois há também o tamanho físico que é obtido com a função len()**

```
def empurra(s, n):
    for i in range(n-1):
        if s[i] > s[i+1]:
            troca(s, i, i+1)
```

Ordenação por flutuação (bubble sort)

- Note que a função empurra –utilizada uma vez, **não ordena a lista** passada como argumento
- Chamando a função empurra uma segunda vez, mas definindo o tamanho lógico da lista com 4, ela empurra o item máximo da lista com quatro itens para a quarta posição, aumentando a parte ordenada para dois itens, e reduzindo a parte desordenada para três itens.
- E assim sucessivamente, chamando-a sempre com um tamanho lógico menor que corresponde à parte não ordenada.
- **Chame a função empurra 4 vezes para ordenar a sequência.**

Bubble sort

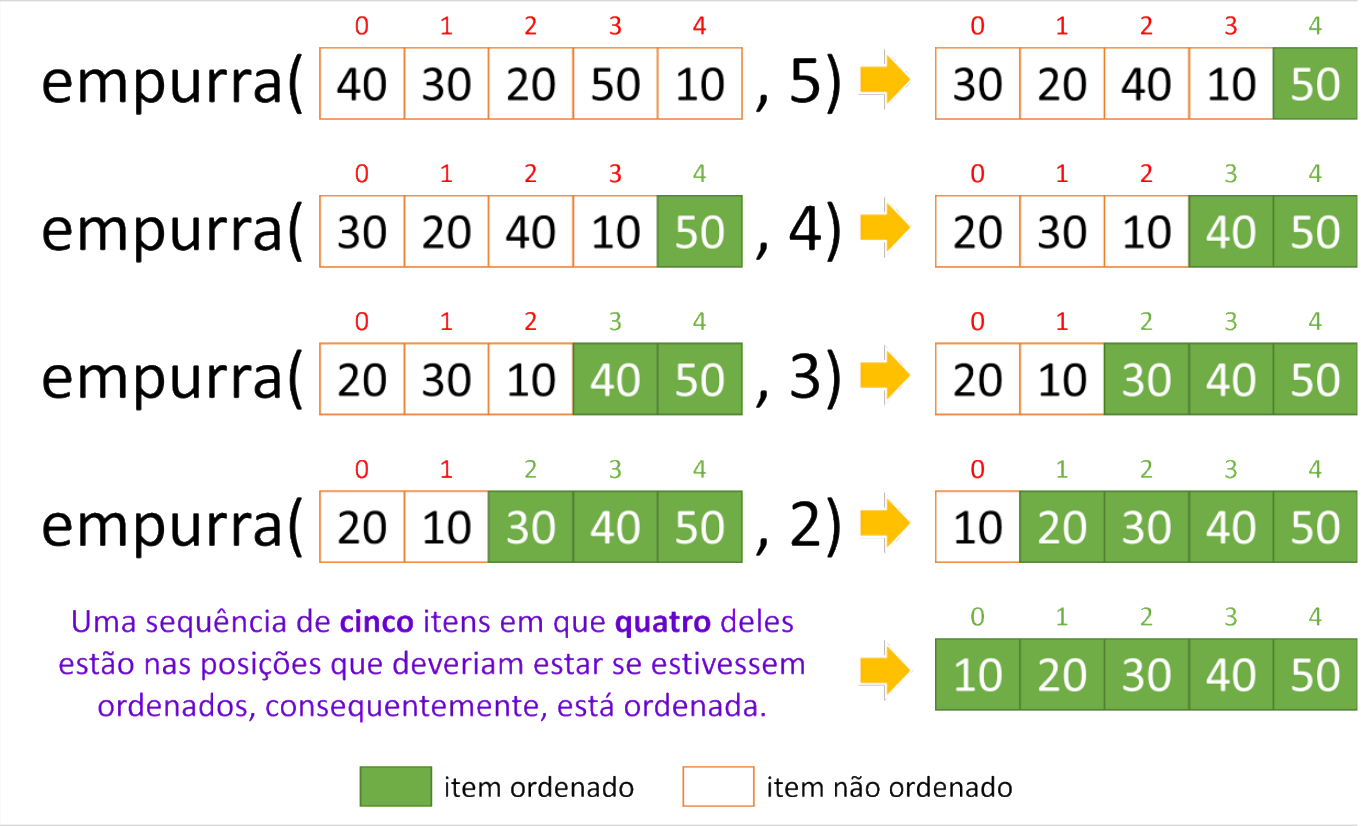


Figura 11.1: Sequência de chamadas à função `empurra`. Fonte: Elaborado pelo autor.

Bubble sort

Criando uma função de ordenação bubble_sort que empurra os itens da sequência mutável s para deixá-la ordenada

```
def bubble_sort(s):
    n = len(s)
    while n > 1:
        empurra(s, n)
        n -= 1
```

Colocando as 3 funções juntas

```
def troca(s, i, j):
    s[i], s[j] = s[j], s[i]

def empurra(s,n):
    for i in range(n-1):
        if s[i] > s[i+1]:
            troca(s, i, i+1)

def bubble_sort(s):
    n = len(s)
    while n > 1:
        empurra(s,n)
        n -= 1
```

Bubble sort

```
>>> lista = [40, 30, 20, 50, 10]
>>> lista
[40, 30, 20, 50, 10]
>>> bubble_sort(lista)
>>> lista
[10, 20, 30, 40, 50]
```

A codificação da função bubble_sort é um exemplo de codificação com **efeito colateral** pois ela **altera** o valor **referenciado pelo parâmetro.**

Ao invés de se ordenar a lista passada como argumento, uma função **sem efeito colateral** retornaria como resposta uma **cópia ordenada da lista original: bubble_sort_2**

Bubble sort

```
def troca(s, i, j):
    s[i], s[j] = s[j], s[i]

def empurra(s,n):
    for i in range(n-1):
        if s[i] > s[i+1]:
            troca(s, i, i+1)

def bubble_sort(s):
    n = len(s)
    while n > 1:
        empurra(s,n)
        n -= 1

def bubble_sort_2(lista):
    lista = lista[:]
    n = len(lista)
    while n > 1:
        empurra(lista, n)
        n -= 1
    return lista
```

Sem efeito colateral

Bubble sort

```
>>> lista = [40, 30, 20, 50, 10]
>>> lista
[40, 30, 20, 50, 10]
>>> nova_lista = bubble_sort_2(lista)
>>> nova_lista
[10, 20, 30, 40, 50]
>>> lista
[40, 30, 20, 50, 10]
```

Recomendações em Python

- Métodos de objetos mutáveis tem efeito colateral e alteram as características do próprio objeto como, por exemplo, a ordem dos itens em uma lista.
- Funções que recebem objetos mutáveis como argumentos não os alteram, retornando um novo objeto gerado a partir do original.

Estabilidade

- Um algoritmo de ordenação é estável quando **elementos iguais mantém a mesma ordem após serem ordenados.**
- Esta é uma característica útil no caso de se fazer ordenações sucessivas na mesma sequência de dados.

Por exemplo: Ordenar uma lista de clientes por nome e depois por data de nascimento de tal modo que clientes com a mesma data de nascimento continuem ordenados alfabeticamente

Alguns algoritmos de ordenação

- **Selection Sort (ordenação por seleção):** percorre a parte não ordenada de uma sequência buscando o maior (ou menor) item e trocando com o último (ou primeiro) da parte não ordenada → aumento da parte ordenada x diminuição da parte não ordenada
- **Insertion Sort (ordenação por inserção):** assume que a parte ordenada da sequência só é composta pelo primeiro item enquanto todos os demais são ordenados. Percorre-se a sequência a partir do primeiro item não ordenado, inserindo-o na parte ordenada até que a parte ordenada fique não vazia.

Alguns algoritmos de ordenação

- **Merge Sort (ordenação por mistura):** divide a sequência sucessivamente até chegar à subsequências contendo um item. Então faz o processo inverso, ou seja, mistura as sequências duas a duas, comparando os itens e colocando-os em ordem
- **Quicksort (ordenação rápida):** escolhe arbitrariamente um item da sequência como pivô ; então a sequência é dividida em duas partes, uma com itens menores que o pivô e outra com itens maiores que ele. Deste modo, o pivô estará na sua posição ordenada mas teremos duas subsequências não ordenadas. O processo é repetido em cada subsequência até que estejam ordenadas.

Alguns algoritmos de ordenação

Quando utilizar e qual tipo de algoritmo utilizar?

- Para sequências pequenas: Selection Sort e Insertion Sort tem o desempenho melhor.
- Para sequências mais parrudas: Merge Sort e Quicksort que requerem mais recursos mas são mais eficientes.

Funções e métodos de ordenação

Função sorted():

```
lista = [40, 30, 20, 10, 50]
lista
[40, 30, 20, 10, 50]
nova_lista = sorted(lista)
nova_lista
[10, 20, 30, 40, 50]
lista
[40, 30, 20, 10, 50]
```

Funções e métodos de ordenação

Para casos em que a sequência original não precise ser alterada, há o método sort que é exclusivo para listas e mais eficiente em consumo de memória pois não a duplica.

```
lista = [40, 30, 20, 50, 10]
lista.sort()
lista
[10, 20, 30, 40, 50]
```

Busca em Python

Quando trabalhamos com coleções - que são estruturas que permitem armazenar itens de dados, não é incomum fazer buscas para verificar se um dado item pertence ou não à coleção. Mesmo para acessar dados associados (dados satélites) ao item buscado, caso o ele seja encontrado.

Busca em Python

Algoritmos de busca:

- **Busca binária:** exigem pré-condições para funcionar**.** Por exemplo, supõe que os itens estejam ordenados em sequências crescentes ou decrescentes**.**
- **Busca linear:** sem pré-condições : o item buscado será comparada com cada item da coleção até ser encontrado ou até que a sequência termine**.**

Busca em Python

Exemplo: Encontrar uma palavra em um dicionário de papel, ou um nome, em uma lista telefônica ou ainda se alguém se lembra.

- Neste caso sabemos que os itens estão em ordem alfabética, coisa que pode nos guiar para uma busca que é mapeada (busca binária).
- E se as palavras estiverem distribuídas aleatoriamente, ou seja, sem qualquer ordem conhecida? Uma solução seria abrir, por exemplo, o dicionário na primeira página e realizar a busca até encontrar a palavra (busca linear ou sequencial).

Busca linear

Função semelhante ao método index: Recebe um valor e uma sequência, devendo retornar o índice da primeira ocorrência do valor na sequência, caso este seja encontrado.

```
def busca_linear(valor, sequencia):
    for i, item in enumerate(sequencia):
        if item == valor:
            return i

    return None
```

Busca Linear

```
def busca_linear(valor, sequencia):
    for i, item in enumerate(sequencia):
        if item == valor:
            return i

    return None
```

A função integrada **enumerate** gera uma **nova sequência** a partir da **sequência argumento.** Cada item da nova sequência é uma tupla com dois valores: um índice e um item da sequência argumento com aquele índice.

O desempacotamento é feito no laço for para, a cada iteração, se ter acesso tanto ao índice quanto ao valor do item. Ao encontrar um item igual ao procurado, a função é encerrada com o retorno do item na sequência. Caso nenhum item seja encontrado que corresponda ao valor procurado, o valor retornado será None.

Busca Binária

A implementação da busca binária é mais elaborada que a da busca linear, mas quando sabemos que a lista está ordenada, este método proporciona desempenho muito superior, que fica mais evidente conforme a busca é realizada em sequências maiores.

Busca Binária – Exemplo-

```
def busca_binaria(valor, sequencia):
    inicio = 0
    fim = len(sequencia) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2

        if valor == sequencia[meio]:
            return meio
        elif valor < sequencia[meio]:
            fim = meio - 1
        else:
            incio = meio + 1

    return None
```

FIM